Lab Assignment 4

What:

In this assignment you are to write a C or Fortran program that parallelizes the application of a convolution operator to an image:

1. Login to prospero via ssh (`ssh <yourusername>@prospero.cs.wit.edu`)
2. Use MPI to parallelize the application of the convolution of the image.
3. Use the provided makefile to build your code. It includes the libraries and include directories that allow you to use OpenCV.
4. Verify that your code produces the correct result.
5. Do **NOT** run the program on the login node, create a batch script and submit to the cluster.
6. Time your code with a variety of ranks.
7. Create a short `PDF` with the content outlined in the Expected Results below. Leave the PDF on the cluster in the same directory where your assignment is.

Why:

Whether for solving physical problems, such as a fluid simulation, or a simple Cellular Automata, like Conway's Game of Life, grid-based applications are ubiquitous in computing. Once one understands the basics breaking up the problem at hand, performing the calculation, and reconstructing the result, this entire class of problems becomes trivial to parallelize.

How:

Using a large image and/or a large kernel is important for this assignment. Find a size that takes at least 5 seconds to run with one rank. This way, the timings you compute will be less influenced by small tasks running on the compute node (i.e. always do timings with a suitably large problem).

The image is small enough that sending the full image to each node will not have much influence on the final time. This will make parallelization easier as each node has all the data.

Outline:

1. Read the image on rank 0
2. Broadcast the image to all ranks
3. Each rank computes its own piece of the blur
4. Reconstruct the full image on rank 0
5. Rank 0 outputs the full image to file

The specifics of how to break up the calculation and reconstruct the final image are up to you. For example, you could have each rank compute the blur for a certain number of rows and perform a gather or gatherv to bring all the results back to rank 0. Or you could do a reduction on the entire image to reconstruct it. Depending on the methods that you choose, there will be slightly different communication timings.

Expected Results:

Create strong scaling plots for:

- The entire runtime of the application
- The total communication time (image distribution and reconstruction)
- The calculation time

Include the three plots, the original image, and the result image in your writeup. The result image should be exactly the same as the result from the serial version of the code.